Informe: Ford-Fulkerson vs. Edmonds-Karp

Taller 9 – Ciencias de la Computación 2

Nombre	Código
Jose Fernando Ramirez Ortiz	20241020080
Joel David Pérez Arroyave	20242020017
Carlos Alberto Barriga Gamez	20222020179
Miguel Ángel Hernández Medina	20222020035

1. Introducción

Ford-Fulkerson y Edmonds-Karp son algoritmos utilizados para calcular el **flujo máximo** de una red dirigida y capacitada. El objetivo consiste en determinar la mayor cantidad de flujo que puede transportarse desde un vértice de origen hasta uno de destino, sin superar la capacidad de las aristas y respetando la conservación del flujo en los vértices intermedios.

Ambos algoritmos utilizan una estructura denominada **red residual**, que representa la capacidad disponible para enviar flujo y permite modificar las decisiones tomadas en iteraciones anteriores.

La diferencia principal entre ellos se encuentra en la estrategia utilizada para encontrar caminos aumentantes: Ford-Fulkerson emplea búsqueda en profundidad (DFS), mientras que Edmonds-Karp utiliza búsqueda en anchura (BFS).

Este informe explica la teoría y la implementación de ambos algoritmos, compara sus complejidades y presenta los resultados obtenidos al ejecutarlos sobre un mismo grafo de ejemplo.

2. Conceptos previos

Antes de explicar los algoritmos, es necesario comprender algunos conceptos utilizados para resolver el problema de flujo máximo.

2.1. Red de flujo

Una red de flujo es un grafo dirigido en el que cada arista tiene una capacidad máxima que representa la cantidad de flujo que puede transportar.

Se identifican dos vértices especiales:

- **Origen (S):** vértice desde el cual comienza el flujo.
- **Destino (T):** vértice al cual se desea enviar la mayor cantidad de flujo posible.

Cada arista tiene una capacidad no negativa. El flujo que circula por ella no puede superar dicha capacidad.

2.2. Camino aumentante

Un camino aumentante es una secuencia de aristas que permite transportar flujo adicional desde el origen hasta el destino. Para que una arista pueda formar parte de este camino, debe tener capacidad residual mayor que cero.

2.3. Red residual

La red residual representa las capacidades que quedan disponibles después de enviar cierta cantidad de flujo.

Cuando se envía flujo por una arista, su capacidad residual disminuye. Al mismo tiempo, aumenta la capacidad residual de la arista inversa, lo que permite cancelar o modificar parte del flujo enviado anteriormente si posteriormente se encuentra un camino más conveniente.

Si una arista tiene capacidad original $c(u, v)$ y se ha enviado un flujo $f(u, v)$, su capacidad residual hacia adelante es:

$$c_f(u, v) = c(u, v) - f(u, v)$$

La capacidad residual hacia atrás permite recuperar flujo previamente enviado. En una representación residual mediante aristas inversas, esta capacidad aumenta con el flujo enviado en sentido contrario.

2.4. Flujo aumentante

Una vez encontrado un camino desde S hasta T, se identifica la menor capacidad residual de sus aristas. Este valor se denomina **flujo aumentante** o cuello de botella:

$$\Delta = \min_{(u,v)\in P} c_f(u, v)$$

Donde P representa el camino encontrado.

El valor Δ indica cuánto flujo adicional puede enviarse por ese camino sin superar ninguna capacidad residual.

3. Implementación de los algoritmos

Los algoritmos se implementaron en clases independientes: `FordFulkerson` y `EdmondsKarp`. Ambos reciben como parámetros el grafo, el vértice de origen y el vértice de destino.

Para trabajar sobre la red, primero se obtiene la lista de vértices y se asigna un índice numérico a cada uno. Posteriormente, se construye una matriz residual de tamaño $V \times V$, en la cual se almacenan las capacidades disponibles entre cada par de vértices.

La matriz se utiliza para consultar rápidamente si existe capacidad residual entre dos vértices y para actualizarla después de cada camino aumentante.

Ambos algoritmos siguen el mismo procedimiento general:

1. Construir la matriz residual a partir de las capacidades originales del grafo.
2. Buscar un camino aumentante desde el origen hasta el destino.
3. Calcular el cuello de botella del camino encontrado.
4. Disminuir la capacidad residual de las aristas utilizadas y aumentar la capacidad residual de las aristas inversas.
5. Sumar el flujo aumentante al flujo máximo acumulado.
6. Repetir el procedimiento hasta que no exista otro camino aumentante.

La diferencia entre las implementaciones se encuentra en el método utilizado para realizar la búsqueda del camino.

Ford-Fulkerson

Ford-Fulkerson es un algoritmo que encuentra el flujo máximo mediante la búsqueda repetida de caminos aumentantes en la red residual. En esta implementación se utiliza **búsqueda en profundidad (DFS)**.

DFS explora un camino avanzando por los vértices disponibles hasta llegar al destino o hasta que no pueda continuar. Si una rama no permite llegar al destino, retrocede y prueba otra posibilidad.

En el programa, la búsqueda se realiza mediante el método `dfs()`, que recibe el vértice actual, el destino, la matriz residual y los arreglos de padres y visitados.

El arreglo de padres permite reconstruir el camino encontrado: cada posición almacena el vértice desde el cual se llegó al vértice actual.

El procedimiento implementado es el siguiente:

1. Se inicializa el arreglo de padres con valores -1 y un arreglo de visitados para evitar recorrer repetidamente los mismos vértices durante una búsqueda.

2. Se ejecuta DFS desde el origen. Para cada vértice se revisan sus posibles destinos y solamente se continúa por aquellos que tengan capacidad residual mayor que cero.

3. Cuando DFS alcanza el destino, se considera que se encontró un camino aumentante. Si no lo alcanza, termina la búsqueda y el algoritmo finaliza.

4. A partir del arreglo de padres se reconstruye el camino desde el destino hasta el origen. Durante este recorrido se obtiene la menor capacidad residual de todas sus aristas.

5. Se actualiza la matriz residual: se resta el flujo aumentante a las aristas del camino y se suma ese mismo valor a las aristas inversas.

6. Se acumula el flujo enviado y se vuelve a ejecutar DFS para buscar otro camino.

Este proceso se repite hasta que DFS no encuentra un camino entre el origen y el destino.

La principal característica de esta implementación es que no establece una preferencia por los caminos más cortos. La ruta seleccionada depende del orden en que se recorren los vértices de la matriz residual. Por esta razón, puede encontrar caminos que requieran más iteraciones para alcanzar el flujo máximo.

Edmonds-Karp

Edmonds-Karp es una variante de Ford-Fulkerson que utiliza **búsqueda en anchura (BFS)** para encontrar los caminos aumentantes.

A diferencia de DFS, BFS explora primero los vértices más cercanos al origen. Para ello utiliza una cola FIFO (*First In, First Out*), en la que los vértices se procesan en el mismo orden en que fueron descubiertos.

En el programa, esta búsqueda se implementa mediante el método `bfs()`, utilizando una cola de tipo `ArrayDeque`.

El procedimiento implementado es el siguiente:

1. Se inicializa el arreglo de padres y se crea una cola para almacenar los vértices pendientes de explorar.

2. Se inserta el origen en la cola y se marca como visitado.

3. Mientras la cola no esté vacía, se extrae el primer vértice y se revisan sus posibles destinos en la matriz residual.

4. Solamente se agregan a la cola los vértices que no hayan sido visitados y cuya capacidad residual sea mayor que cero. Al descubrir un vértice se registra su padre.

5. Si se alcanza el destino, se detiene la búsqueda porque BFS ya encontró un camino con el menor número de aristas entre los caminos disponibles.

6. Se reconstruye el camino utilizando el arreglo de padres y se calcula su menor capacidad residual.

7. Se actualizan las capacidades residuales, se acumula el flujo aumentante y se vuelve a ejecutar BFS.

El algoritmo finaliza cuando BFS ya no puede encontrar un camino desde el origen hasta el destino.

La ventaja de esta estrategia es que selecciona caminos con el menor número de aristas, lo que permite establecer una cota polinomial para la cantidad de caminos aumentantes. De esta manera, Edmonds-Karp tiene un comportamiento teórico más predecible que la versión general de Ford-Fulkerson.

Registro de resultados

Para comparar ambos algoritmos se implementó una clase `ResultadoFlujo`, que almacena las métricas obtenidas durante cada ejecución:

- **Flujo máximo:** cantidad total de flujo enviado desde el origen hasta el destino.
- **Tiempo de ejecución:** tiempo medido en nanosegundos y presentado en milisegundos.
- **Iteraciones:** cantidad de caminos aumentantes utilizados para incrementar el flujo.
- **Inspecciones de aristas:** cantidad de posiciones de la matriz residual revisadas durante las búsquedas.

La clase `ComparadorFlujo` ejecuta ambos algoritmos sobre el mismo grafo y presenta sus resultados en una tabla. De esta manera, se pueden comparar tanto el resultado obtenido como el trabajo realizado para alcanzarlo.

4. Grafo de ejemplo

Para evaluar los algoritmos se utilizó una red dirigida compuesta por 13 vértices y 24 aristas. El vértice S representa el origen y el vértice T representa el destino. Los valores asociados a las aristas corresponden a sus capacidades.

La Figura 1 presenta la estructura de la red utilizada en las pruebas.

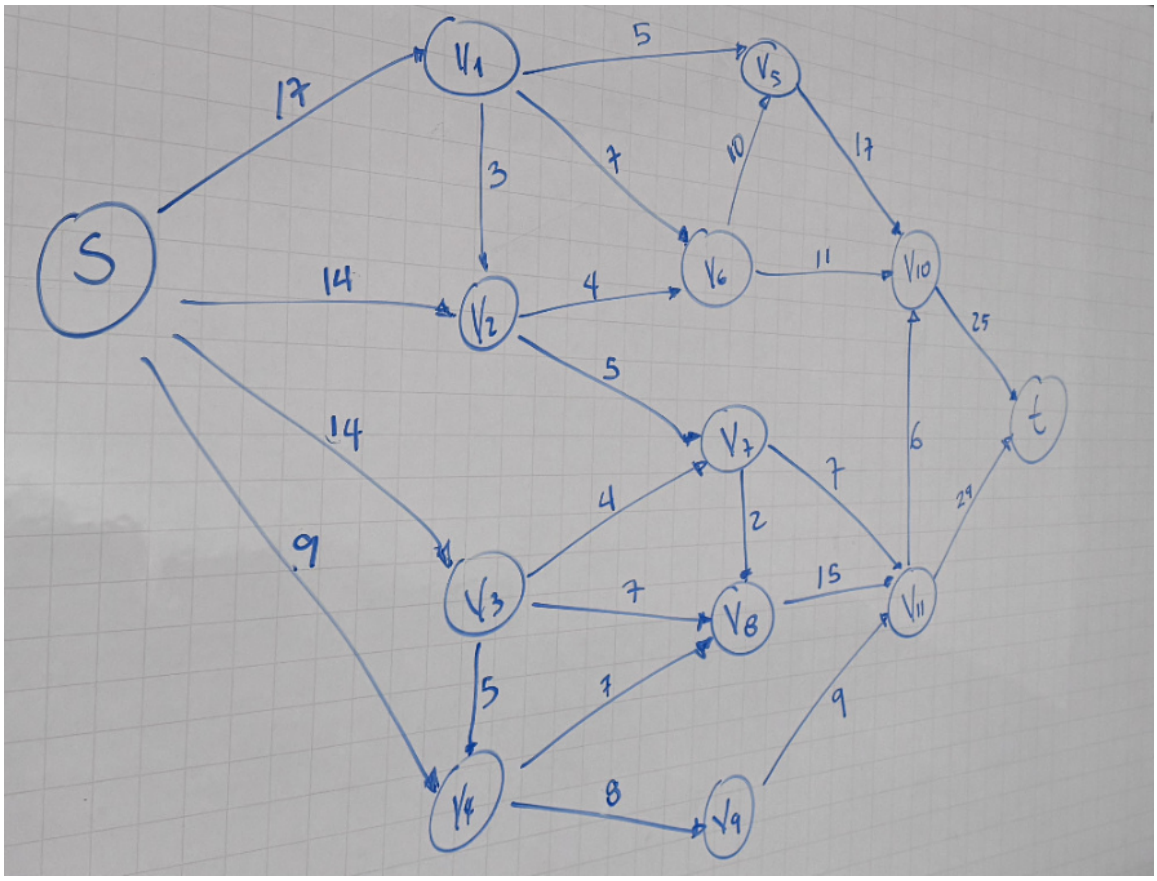


Figura 1: Grafo dirigido utilizado para comparar Ford-Fulkerson y Edmonds-Karp.

5. Comparación

5.1. Comparación general

	Ford-Fulkerson	**Edmonds-Karp**
Estrategia de búsqueda	Búsqueda en profundidad (DFS).	Búsqueda en anchura (BFS).
Selección del camino	Selecciona un camino según el orden de exploración.	Selecciona un camino con el menor número de aristas.
Complejidad temporal	$O(EF)$ para capacidades enteras.	$O(VE^2)$.
Complejidad espacial teórica	$O(V + E)$ con listas de adyacencia.	$O(V + E)$ con listas de adyacencia.
Complejidad espacial en este proyecto	$O(V^2)$ por la matriz residual.	$O(V^2)$ por la matriz residual.
Iteraciones máximas	$O(F)$ para capacidades enteras.	$O(VE)$.

Donde V representa el número de vértices, E el número de aristas y F el valor del flujo máximo.

5.2. Estrategia de búsqueda

La diferencia principal se encuentra en la forma de recorrer la red residual:

- **Ford-Fulkerson:** DFS explora una rama hasta donde sea posible antes de retroceder. No garantiza caminos cortos y el resultado de la búsqueda depende del orden de exploración.

- **Edmonds-Karp:** BFS explora por niveles y encuentra primero los vértices más cercanos al origen. Por ello, selecciona caminos aumentantes con el menor número de aristas.

5.3. Complejidad temporal

La complejidad temporal expresa cómo crece el tiempo de ejecución cuando aumenta el tamaño de la red.

- **Ford-Fulkerson:** para capacidades enteras no negativas, su complejidad es $O(EF)$. En el peor caso, puede necesitar hasta F caminos aumentantes y cada búsqueda tiene un costo proporcional a E utilizando listas de adyacencia.
- **Edmonds-Karp:** su complejidad es $O(VE^2)$. Cada búsqueda BFS tiene un costo de $O(V+E)$ y el número de caminos aumentantes está acotado por $O(VE)$.

Edmonds-Karp tiene una cota polinomial que no depende del valor numérico del flujo máximo. En cambio, la complejidad de Ford-Fulkerson depende de F cuando las capacidades son enteras.

5.4. Complejidad espacial

La complejidad espacial representa la memoria adicional requerida durante la ejecución.

En una implementación basada en listas de adyacencia, ambos algoritmos pueden trabajar con un espacio de $O(V + E)$ para almacenar la red residual y las estructuras auxiliares de búsqueda.

Sin embargo, en este proyecto se utiliza una matriz residual de dimensiones $V \times V$. Por lo tanto, esta estructura requiere $O(V^2)$ de memoria.

Adicionalmente, se utilizan arreglos de padres y visitados, así como una pila de recursión para DFS o una cola para BFS. Estas estructuras requieren $O(V)$ de memoria.

En consecuencia, la complejidad espacial de ambas implementaciones es:

$$O(V^2)$$

5.5. Iteraciones máximas

Las iteraciones representan la cantidad de caminos aumentantes encontrados antes de alcanzar el flujo máximo.

- **Ford-Fulkerson:** cuando las capacidades son enteras, cada aumento transporta al menos una unidad de flujo. Por ello, el número de iteraciones está acotado por F. Para capacidades no enteras, esta cota no se aplica y la versión general puede no terminar con determinadas elecciones de caminos.
- **Edmonds-Karp:** el número de caminos aumentantes está acotado por $O(VE)$, independientemente del valor numérico del flujo máximo.

En el grafo utilizado se tienen $V = 13$, $E = 24$ y un flujo máximo de $F = 44$. Por tanto, la cota de Ford-Fulkerson para este caso con capacidades enteras es de 44 aumentos. Para Edmonds-Karp, el producto VE es 312, lo cual permite expresar su cota como $O(312)$ para este grafo, sin que esto signifique que necesariamente realizará esa cantidad de iteraciones.

5.6. Resultados de la medición

La comparación se realizó ejecutando ambos algoritmos sobre el grafo de la Figura 1, utilizando S como origen y T como destino.

Los resultados obtenidos fueron los siguientes:

Métrica	**Ford-Fulkerson**	**Edmonds-Karp**
Flujo máximo	44,0000	44,0000
Tiempo (ms)	0,557000	0,359100
Iteraciones	19	9
Inspecciones de aristas	1667	1170

5.7. Análisis de los resultados

- **Flujo máximo:** ambos algoritmos obtuvieron 44 unidades de flujo, por lo que coinciden en la solución para la red evaluada.
- **Tiempo de ejecución:** Ford-Fulkerson tardó 0,557000 ms, mientras que Edmonds-Karp tardó 0,359100 ms. En esta ejecución, Edmonds-Karp fue aproximadamente 1,55 veces más rápido.
- **Iteraciones:** Ford-Fulkerson necesitó 19 caminos aumentantes y Edmonds-Karp solamente 9. Esto representa una reducción aproximada del 52,63 % en el número de iteraciones.
- **Inspecciones de aristas:** Ford-Fulkerson realizó 1667 inspecciones, mientras que Edmonds-Karp realizó 1170. Esto representa aproximadamente un 29,81 % menos de inspecciones para Edmonds-Karp.

5.8. Casos donde son similares

- **Resultado:** ambos algoritmos obtuvieron el mismo flujo máximo de 44 unidades.
- **Objetivo:** ambos utilizan caminos aumentantes y actualizan la red residual hasta que no es posible enviar más flujo.
- **Memoria:** en esta implementación utilizan la misma estructura principal, una matriz residual de tamaño $V \times V$, por lo que tienen el mismo orden de complejidad espacial.

5.9. Casos donde son diferentes

En la ejecución realizada, Edmonds-Karp obtuvo mejores resultados en las tres métricas de rendimiento medidas:

Métrica	**Ford-Fulkerson**	**Edmonds-Karp**
Tiempo (ms)	0,557000	0,359100
Iteraciones	19	9
Inspecciones	1667	1170

Las diferencias pueden explicarse por la estrategia de búsqueda utilizada:

- Ford-Fulkerson puede seleccionar caminos que no sean los más cortos, dependiendo del orden de exploración de DFS.
- Edmonds-Karp selecciona caminos con el menor número de aristas, lo que en este caso permitió alcanzar el flujo máximo con menos iteraciones.

- Al realizar menos iteraciones e inspecciones, Edmonds-Karp también registró un menor tiempo de ejecución.

Estos resultados corresponden a una ejecución sobre un grafo pequeño. Los tiempos pueden variar según el equipo, la carga del sistema y las condiciones de medición. Por ello, no se puede concluir que Edmonds-Karp siempre sea más rápido para cualquier red.

5.10. ¿Cuándo usar cada uno?

- **Ford-Fulkerson:** resulta conveniente cuando se busca una implementación sencilla y se trabaja con capacidades enteras. Puede ser eficiente si la selección de caminos es favorable.
- **Edmonds-Karp:** es conveniente cuando se necesita una cota temporal polinomial y un comportamiento más predecible, independientemente del valor del flujo máximo.
- **Ambos:** permiten resolver el problema de flujo máximo y deben producir el mismo resultado cuando se ejecutan correctamente sobre la misma red.

6. Conclusión

Ford-Fulkerson y Edmonds-Karp permiten calcular el flujo máximo de una red dirigida mediante caminos aumentantes y una red residual. Su principal diferencia se encuentra en la estrategia de búsqueda: Ford-Fulkerson utiliza DFS, mientras que Edmonds-Karp utiliza BFS.

En la prueba realizada, ambos obtuvieron un flujo máximo de 44 unidades. Sin embargo, Edmonds-Karp necesitó 9 iteraciones y 1170 inspecciones de aristas, frente a las 19 iteraciones y 1667 inspecciones de Ford-Fulkerson. También registró un menor tiempo de ejecución: 0,359100 ms frente a 0,557000 ms.

Por lo tanto, Edmonds-Karp presentó un mejor rendimiento en esta prueba. Esto se relaciona con su estrategia de seleccionar caminos con el menor número de aristas y con su cota polinomial de complejidad temporal.

No obstante, una sola medición sobre un grafo pequeño no permite establecer que siempre será más rápido. La elección depende del tamaño de la red, las capacidades y las condiciones de ejecución.